

A Systolic Array-based CNN Accelerator for Gesture Recognition

Team 5: LIC Lab
Instructor: Prof. Harish Krishnaswamy
Teaching Assistant: Alfred Davidson
Group member: Chengcheng Xu(cx2355); Linxiao Wu(lw3227); Yizheng Tang(yt2992); Cheng-Wen Chu(cc5397)
Date:05/01/2026

What's New in This Milestone

Category	Update
Area Optimization	Our newest version of system reduce 7% of netlist area -> Density after PNR is also reduced
System Verification (Behavior)	Verified the system with 1600+ images, 0 error
System Verification (Post PNR @50MHz)	Post-PnR sim passes at all three SDF corners (MAX / TYP / MIN); with zero timing violations.
DF design(SCAN CHAIN)	Scan Chain + FSM
IO PINS	Our chip has 44 pads in total, including signal IOs, power/ground pads, and Scan chain.
Updated PNR	DFT module + Core
FPGA Verification	The same RTL was synthesized and programmed onto an FPGA

Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

04 Scan Chain

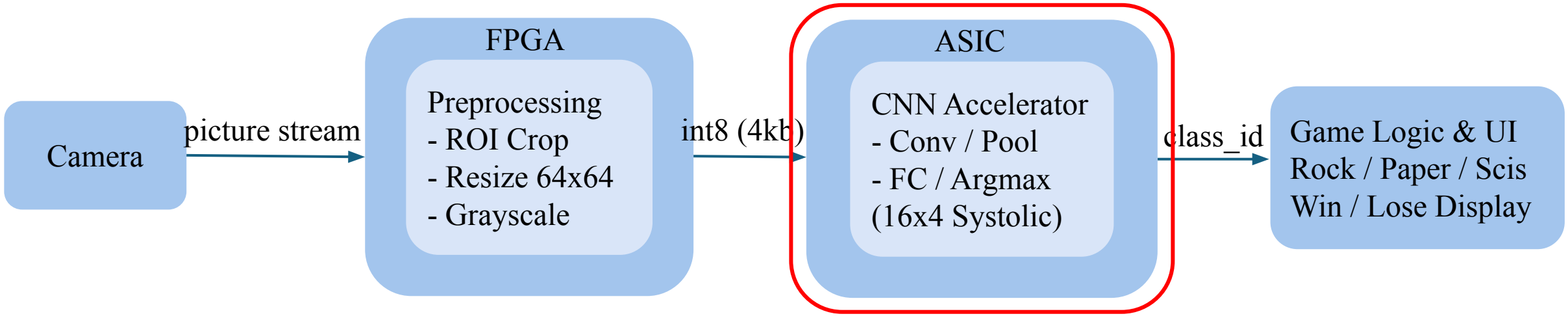
05 Digital PNR Results

06 Verification

07 Summary

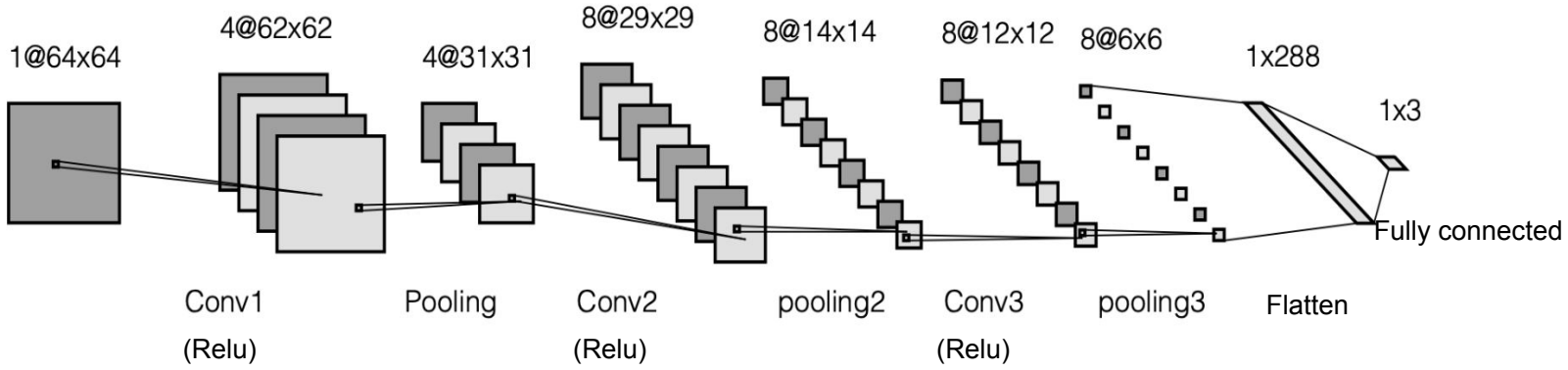
Introduction

Overall System Architecture



Block	Location	Primary Function	Design Purpose / Benefit
FPGA Preprocessing Pipeline	Off-chip (FPGA)	• ROI cropping • Resize to 64×64 • Grayscale & normalization	• Reduces input data volume • Offloads non-compute tasks from ASIC • Simplifies on-chip CNN design
CNN inference	On-chip (ASIC)	• 3-layer Conv + Quant + Pool pipeline (4×16 systolic array) • 3-class FC classifier • Dual-bank SRAM ping-pong buffering • Autonomous FSM sequencing (preload → L1 → L2 → L3 → FC)	• Single-chip end-to-end int8 inference, no off-chip memory access between layers • Fused Conv→Quant→Pool minimizes intermediate storage • Valid/ready streaming interface — host loads once, chip infers autonomously

CNN Structure



3 Layers of Convolution

Kernel size：3x3

conv1: 3x3@4 conv2: 3x3@8 conv3: 3x3@8

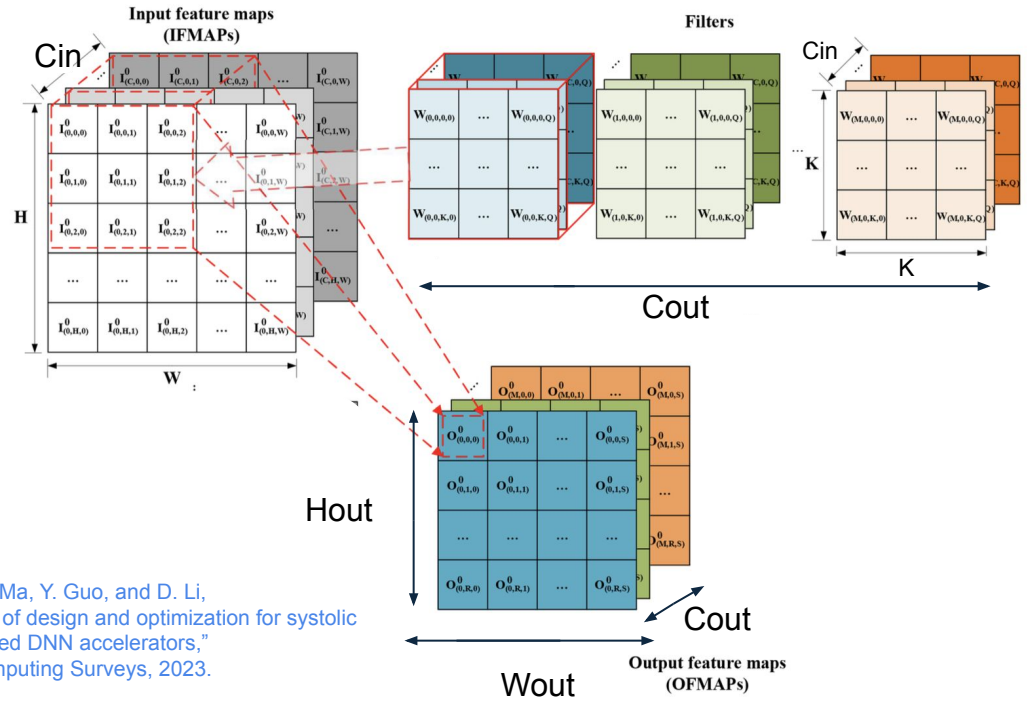
MAC operation for convolution per image:

MACs=Wout x Hout x Cout x (K x K x Cin)

	W	H	Cout	*KxKxCin*	
conv1:	62	x 62 x	4	x 9	=138384
conv2:	29	x 29 x	8	x 36	=242208
conv3:	12	x 12 x	8	x 72	=82944

Activation Function:

ReLU(x) = max(0, x)



R. Xu, S. Ma, Y. Guo, and D. Li,
"A survey of design and optimization for systolic array-based DNN accelerators,"
ACM Computing Surveys, 2023.

Pooling

Size: 2x2, Stride: 2

- Reduces spatial size by 2×
- Keeps channel number unchanged
- Reduces computation of next layer

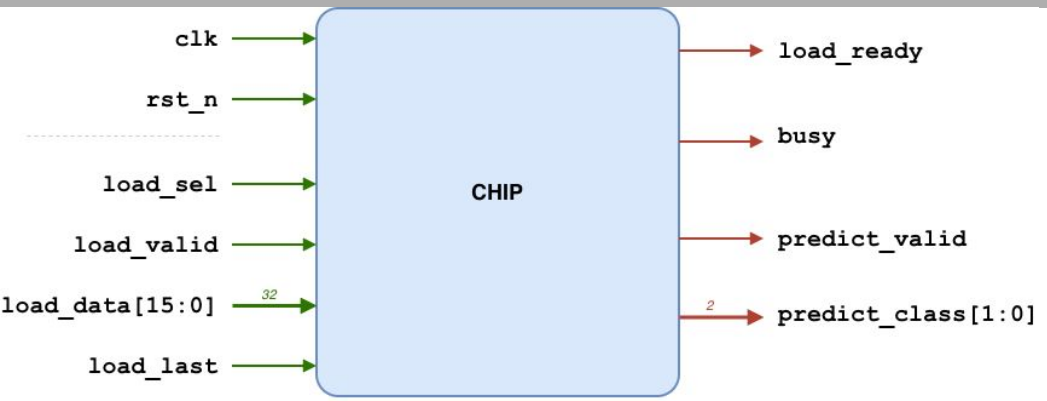
Flatten:

- Reshapes 3D feature maps to 1D vector
- No computation (only data reordering)
- No MAC required
- Memory access only

Fully Connection:

- Dense matrix multiplication
- Each output neuron connects to all inputs
- MACs = Nin × Nout
- *288 × 3 = 864 MACs*

IO Configuration



Signal Pad: 26
Power Pad: 13
DFT Pad : 5
Total Pad : 44

Pin Name	Direction	Description
CLK	In	External System Clock input
rst_n	In	Active-low asynchronous reset
load_sel	In	0: Load parameters of CNN model from the host; 1: Load 1 Image from the host
load_valid	In	Valid_in Siganl
load_data[15:0]	In	The only data bus, carries packed data from the external host
load_last	In	load last beat indicator
load_ready	Out	indicates that the system can accept data from the host
busy	Out	Chip busy flag. While busy is asserted, the host should not start a new load transaction
predict_valid	Out	Indicates that predict_class holds a valid classification output.
predict_class[1:0]	Out	Predicted class index (0–2).

Host Communication Protocol

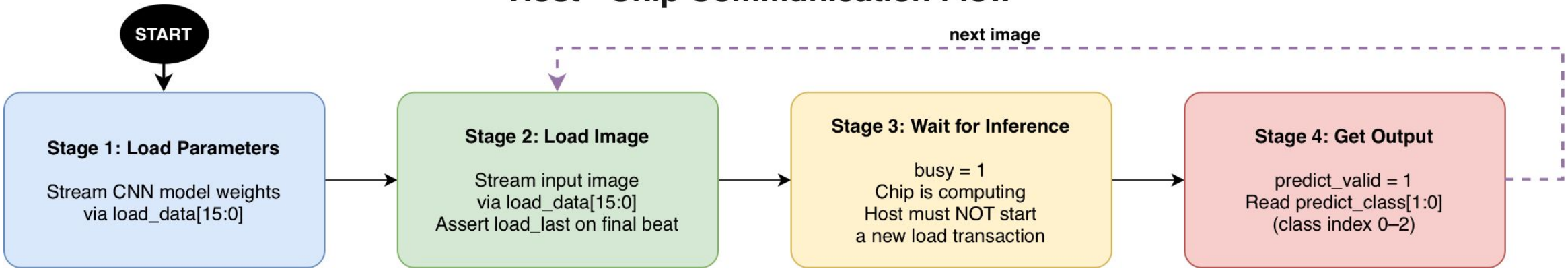
The chip communicates with an external host via a simple **valid/ready handshake** on a shared 16-bit data bus (`load_data[15:0]`).

The host selects the transfer mode using `load_sel`:

- `load_sel = 0` → load CNN model parameters
- `load_sel = 1` → load input image data

Host Communication Protocol

Host - Chip Communication Flow



From Host

Load_sel	0
Load_valid	1
Load_last	last_data?1:0
Load_data	Loading Parameters

From Chip

Load_ready	1
busy	0
predict_valid	0
predict_class	Don't care

From Host

Load_sel	1
Load_valid	1
Load_last	last_data?1:0
Load_data	Loading images

From Chip

Load_ready	1->0->1->....
busy	0
predict_valid	0
predict_class	Don't care

From Host

Load_sel	Don't care
Load_valid	0
Load_last	Don't care
Load_data	Don't care

From Chip

Load_ready	0
busy	1
predict_valid	0
predict_class	Don't care

From Host

Load_sel	Don't care
Load_valid	0
Load_last	Don't care
Load_data	Don't care

From Chip

Load_ready	0
busy	0
predict_valid	1
predict_class	R/P/S

Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

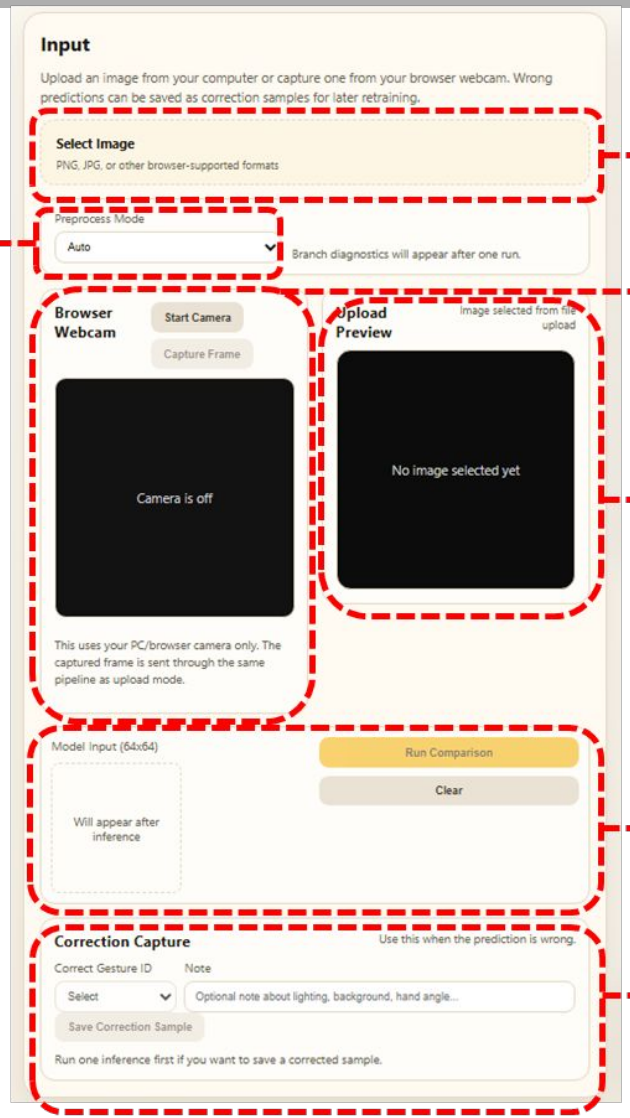
04 Scan Chain

05 Digital PNR Results

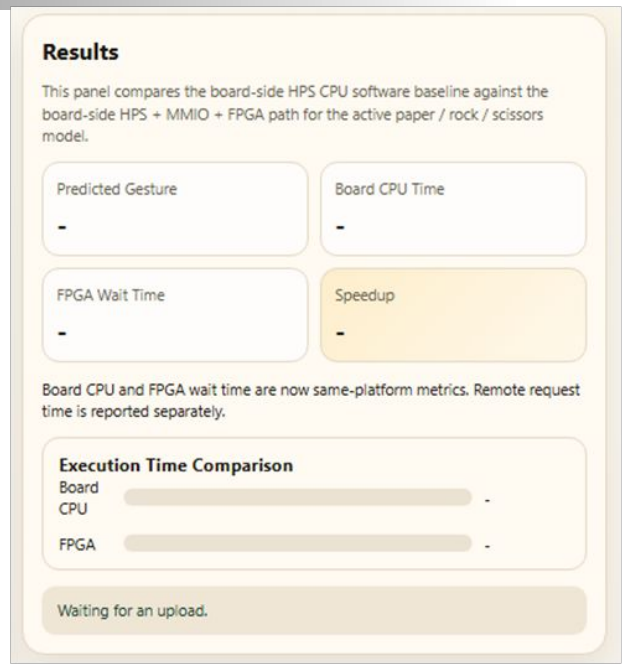
06 Verification

07 Summary

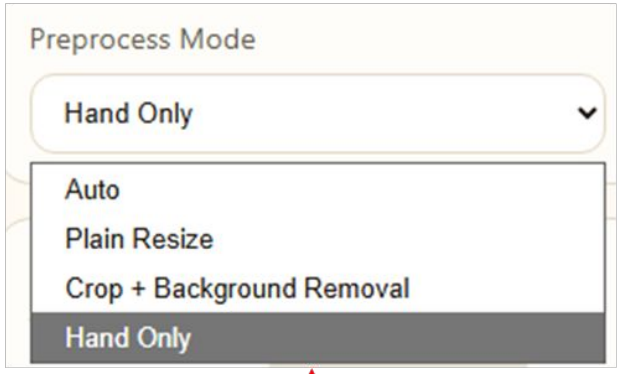
Website introduce



- **Select Image:**
Allows users to upload a local gesture image, preview it on the web page, and use it as input for CPU and FPGA inference.
- **Browser Webcam:**
Captures a gesture frame directly from the user's webcam and sends it to the same CPU and FPGA inference pipeline.
- **Upload Preview:**
Displays the selected or captured image before preprocessing, allowing users to verify the input before running inference.
- **Model Input:**
Shows the preprocessed 64×64 image that is converted to INT8 format and sent to the CPU and FPGA inference paths.
- **Correction Capture:**
Allows users to save misclassified samples with the correct label for future dataset improvement and model retraining.



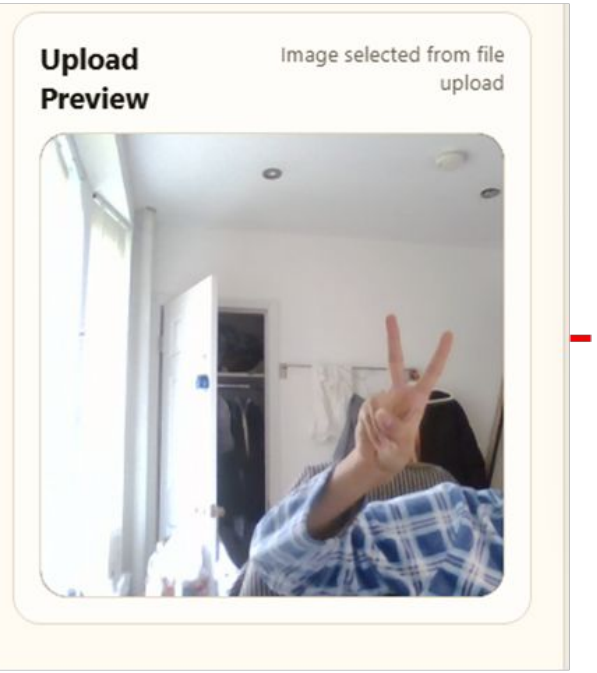
- **Results:**
Displays the predicted gesture and compares board CPU time, FPGA wait time, and speedup after inference.



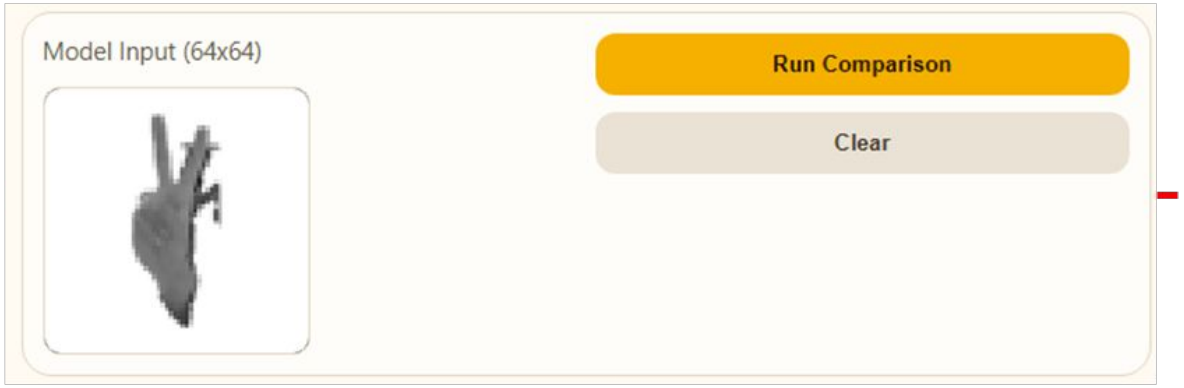
- **Preprocess Mode:**
Lets users choose how the input image is processed before inference, such as plain resize, background removal, hand-only extraction, or automatic selection.

Preprocessing & Feedback

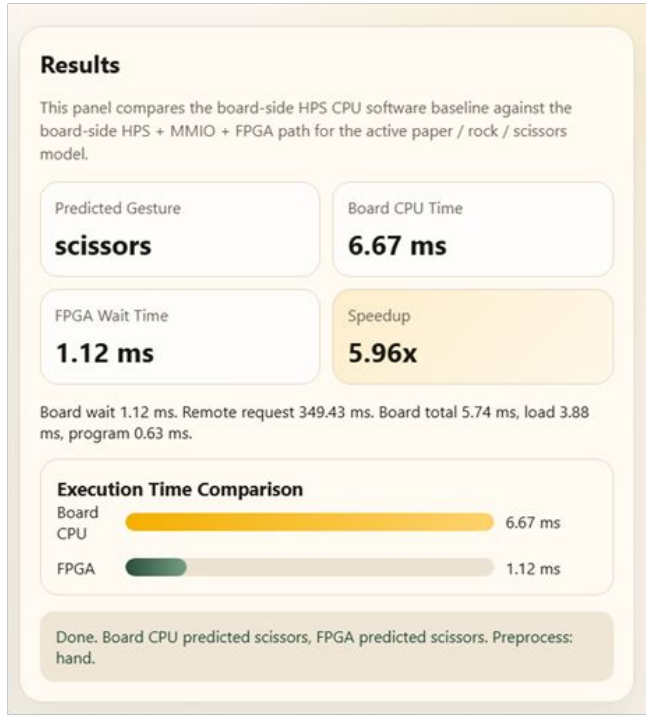
Captured gesture image



Preprocessed Image

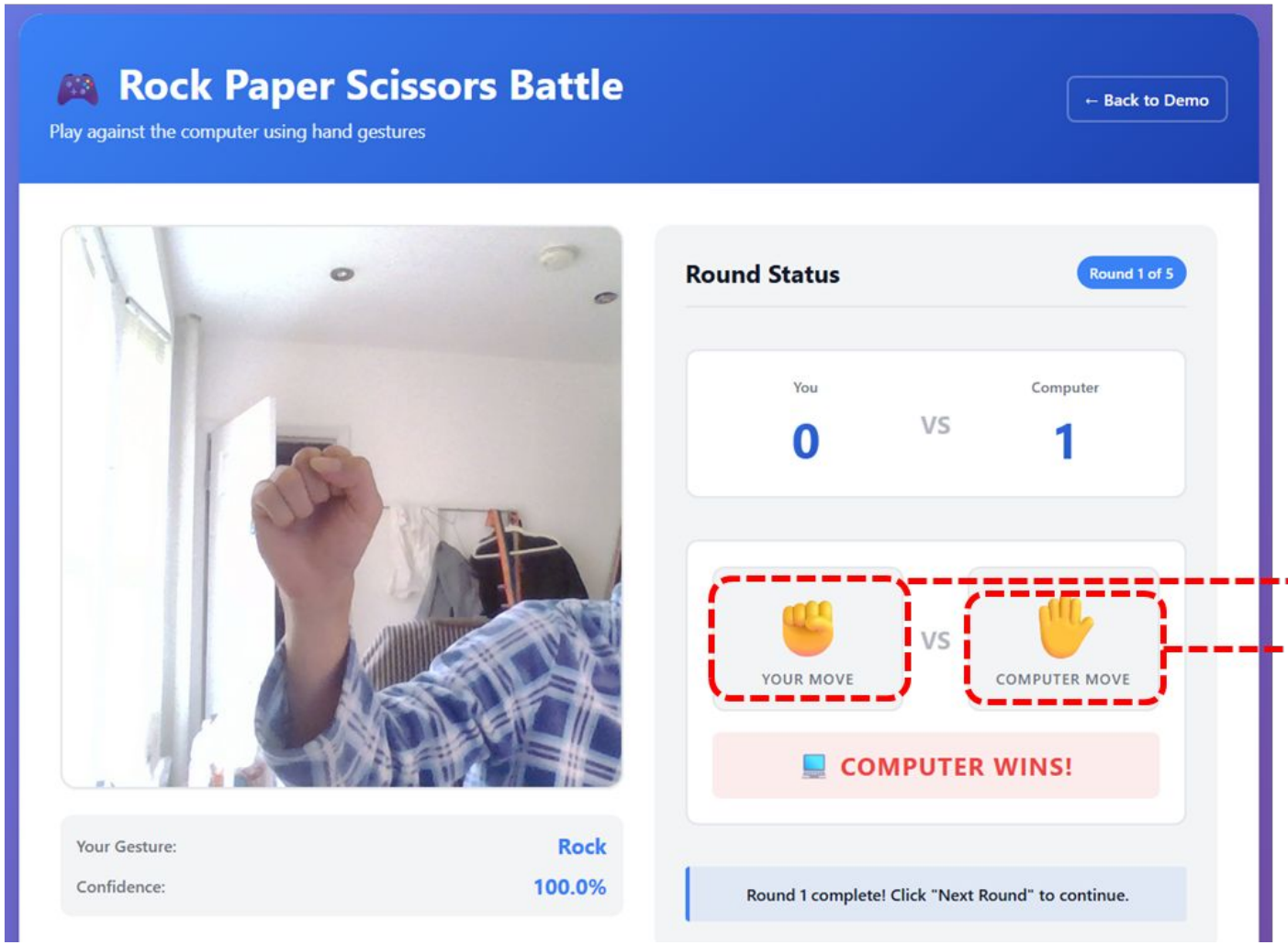


Compare with CPU onboard



This flow demonstrates the preprocessing and feedback workflow. A captured gesture image is first uploaded or taken from the webcam, then converted into a 64×64 preprocessed model input with background removal and hand-region extraction. The processed image is sent to both the onboard CPU baseline and the FPGA accelerator, allowing the website to compare prediction results, execution time, and speedup.

Game demo

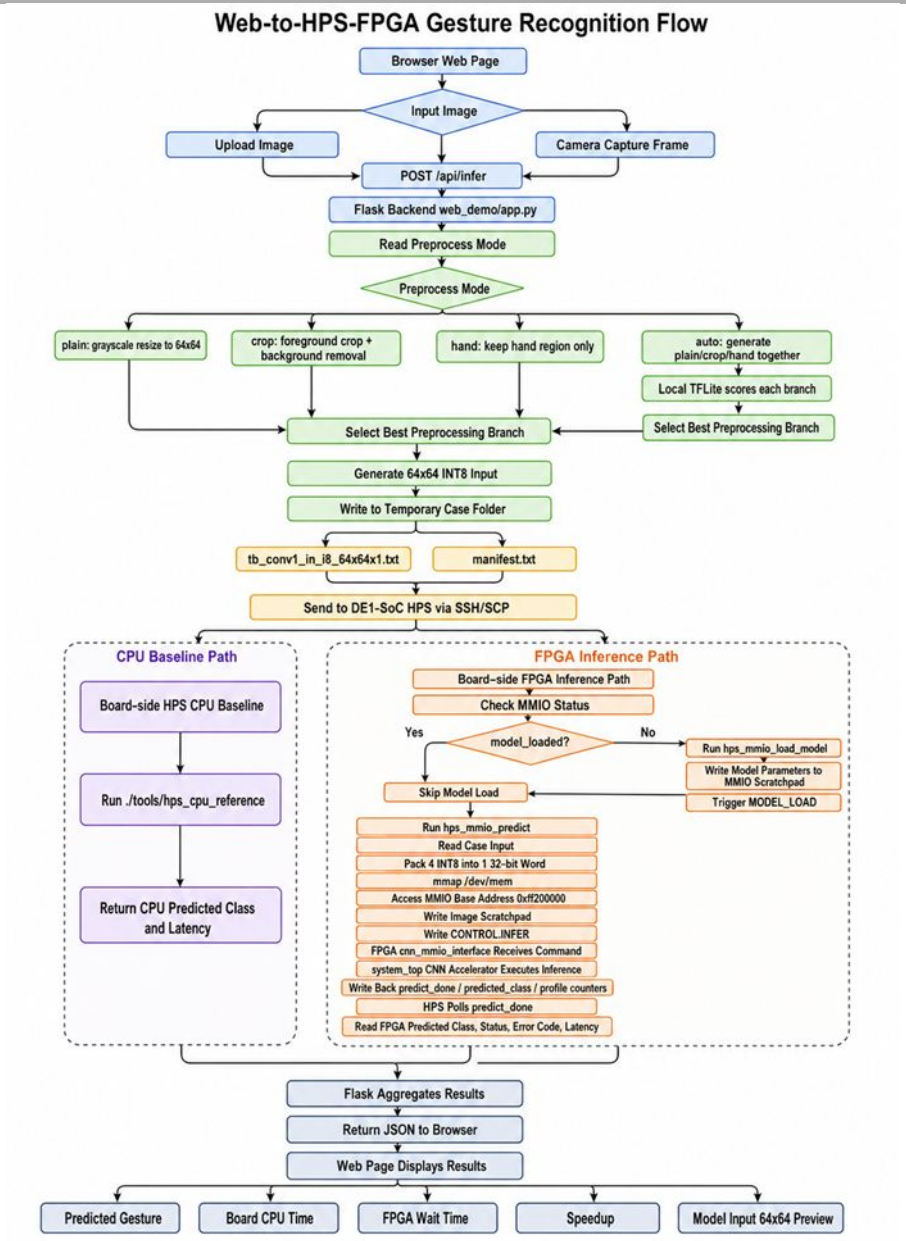


- The game uses the browser webcam to capture the player's hand gesture during each round. The captured frame is sent to the backend, where the CNN model classifies it as rock, paper, or scissors and returns the predicted gesture with confidence. The computer then randomly selects its move, and the game compares both choices using standard rock-paper-scissors rules. Scores are updated after each round, and the match continues until one side wins the best-of-five game.

Capture your gesture

Random from computer

Web-to-FPGA Inference Pipeline



- This flow shows how the web demo connects user input to FPGA inference. An image from upload or webcam capture is sent to the Flask backend, where it is preprocessed into a 64×64 INT8 tensor using plain resize, background removal, hand-only extraction, or automatic branch selection. The processed case is then transferred to the DE1-SoC HPS through SSH/SCP.
- On the board, the HPS runs both a CPU baseline and the FPGA path. For FPGA inference, the HPS loads the model if needed, writes the packed input tensor through MMIO at 0xff200000, triggers the accelerator, and reads back the predicted class, status, error code, and timing. Finally, Flask aggregates the CPU and FPGA results and returns them to the browser for display.

Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

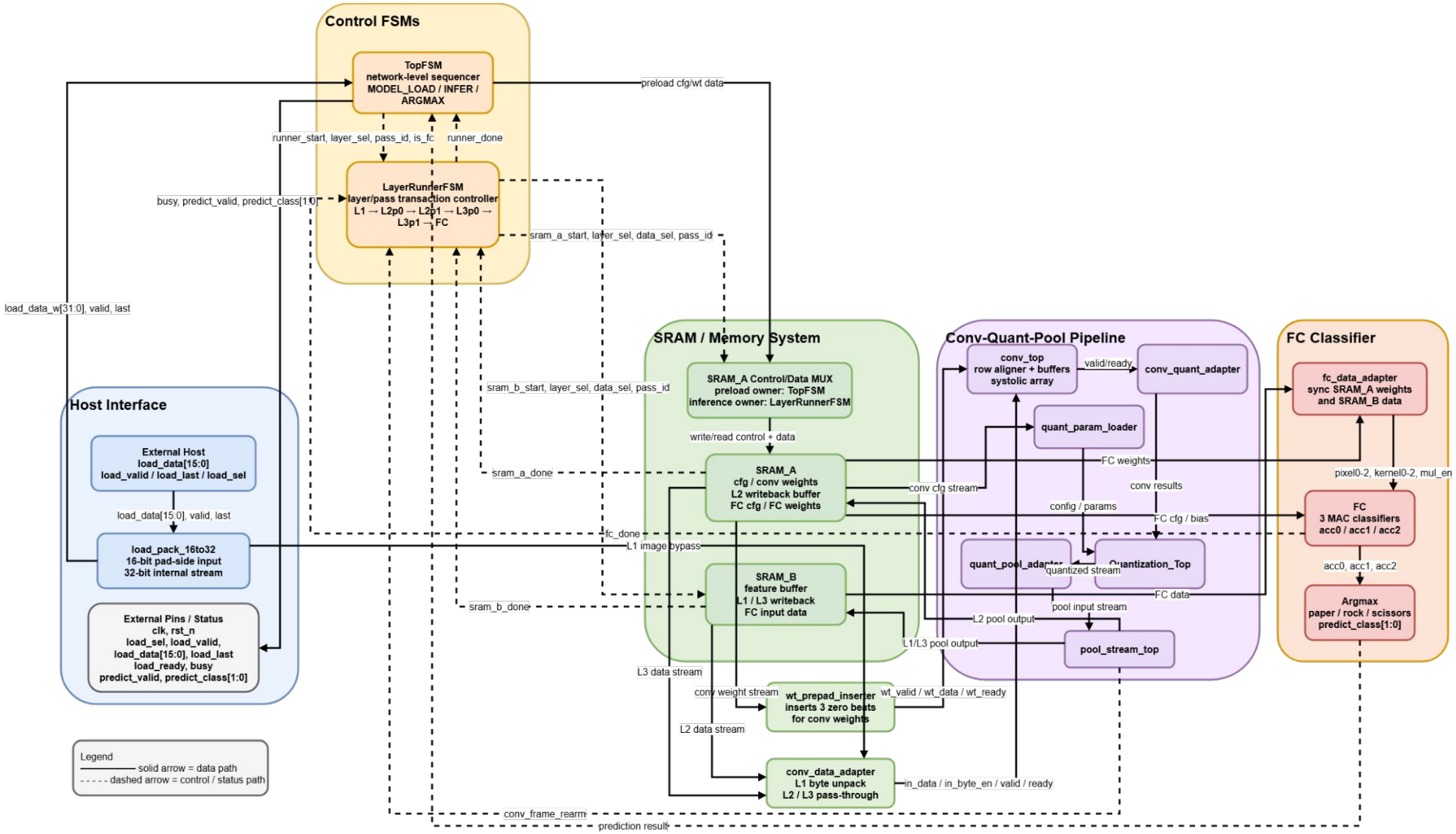
04 Scan Chain

05 Digital PNR Results

06 Verification

07 Summary

Block diagram



Data flow

Stage	Input Source	Processing Path	Output / Destination	Purpose
Model Load	Host	load_pack_16to32	SRAM_A	Load model cfg and weights
L1	Host image	load_pack_16to32 -> conv_data_adapter -> Conv-Quant-Pool	SRAM_B	First conv layer, image bypass
L2	SRAM_B	conv_data_adapter -> Conv-Quant-Pool	SRAM_A	Second conv layer, write back to SRAM_A
L3	SRAM_A	conv_data_adapter -> Conv-Quant-Pool	SRAM_B	Third conv layer, write back to SRAM_B
FC	SRAM_A weights/cfg + SRAM_B data	fc_data_adapter / FC -> Argmax	predict_class[1:0]	Final classification

Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

04 Scan Chain

05 Digital PNR Results

06 Verification

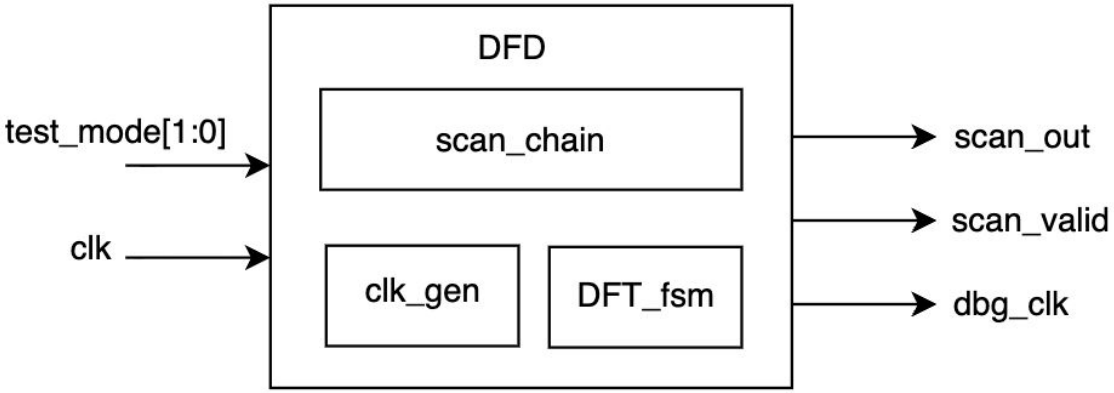
07 Summary

Pipeline Design for Debug

Why DFD

Item	Full scan (DFT)	This work (DFD)
Primary goal	Manufacturing fault testing, structural coverage	Post-silicon debug, observing CNN pipeline behavior
Observation scope	All scan flip-flops (global state dump)	Only selected internal nodes (POOL / QNT / SA outputs)
Effect of scan-out	Shifts and destroys captured state; must rerun workload to see the next packet	Captures one packet of signals of interest into a debug chain; focuses on outputs, not full state
Data per snapshot	Very large; long scan-out time	Small packet (layer/sample + one stage output); short scan-out time
Interpretation	Hard: raw FF bits → CNN semantics	Easy: packet fields map directly to layer / sample / stage
Best suited for	Production test	Lab bring-up and CNN pipeline debugging

Pipeline Design for Debug

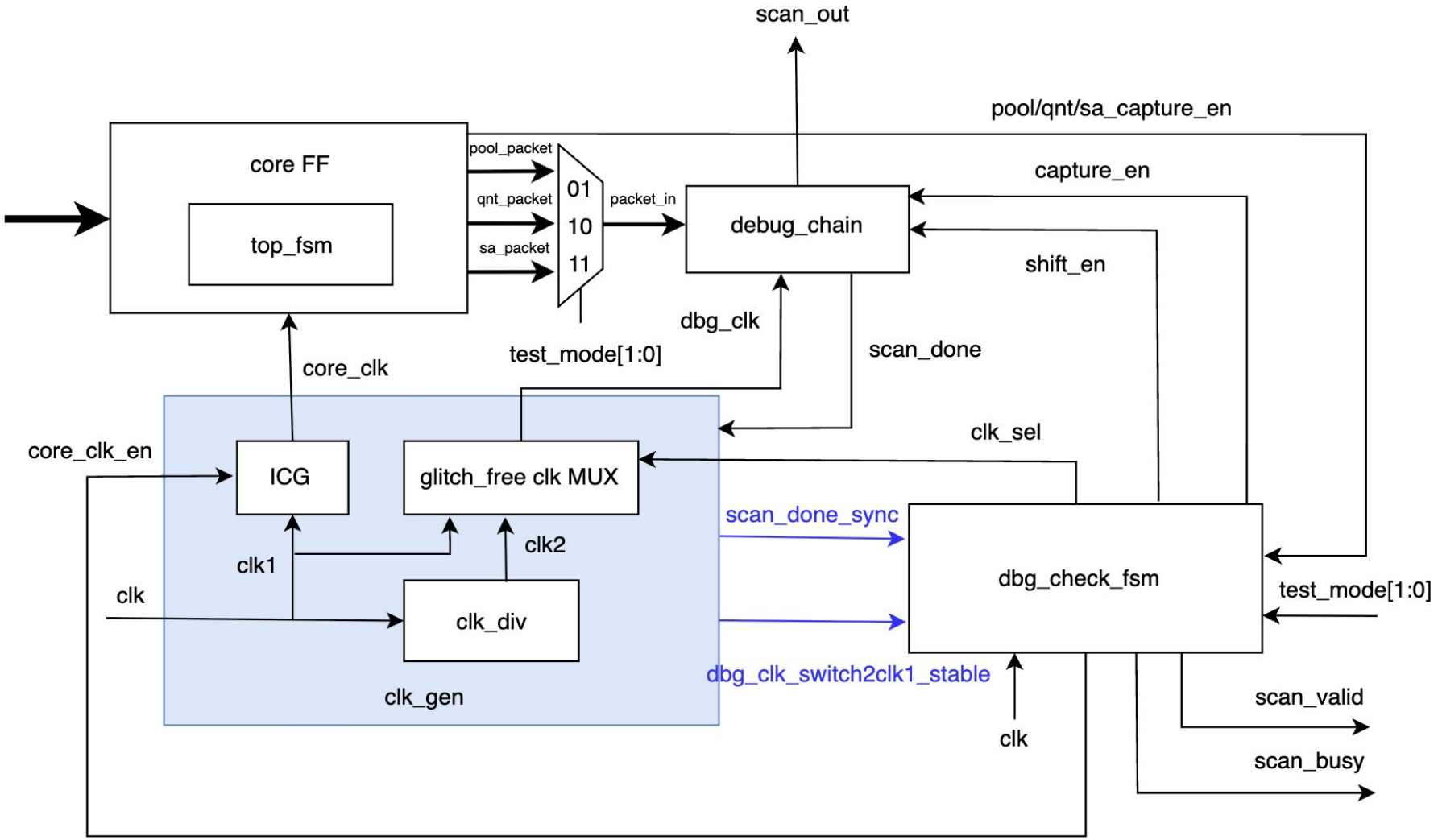


Features:

- Automated self-triggered scan-out DFD
- Selective debug modes with unified 102-bit scan packets
- Pause-capture-shift-resume scan FSM operation
- Glitch-free clocking using ICG-based core clock gating and a glitch-free clock MUX
- Source-synchronous scan-out protocol using dbg_clk, scan_valid, and scan_out
- FPGA-side logging of all 102-bit scan-out packets
- Python-based comparison against MATLAB golden data with fully correct verification results

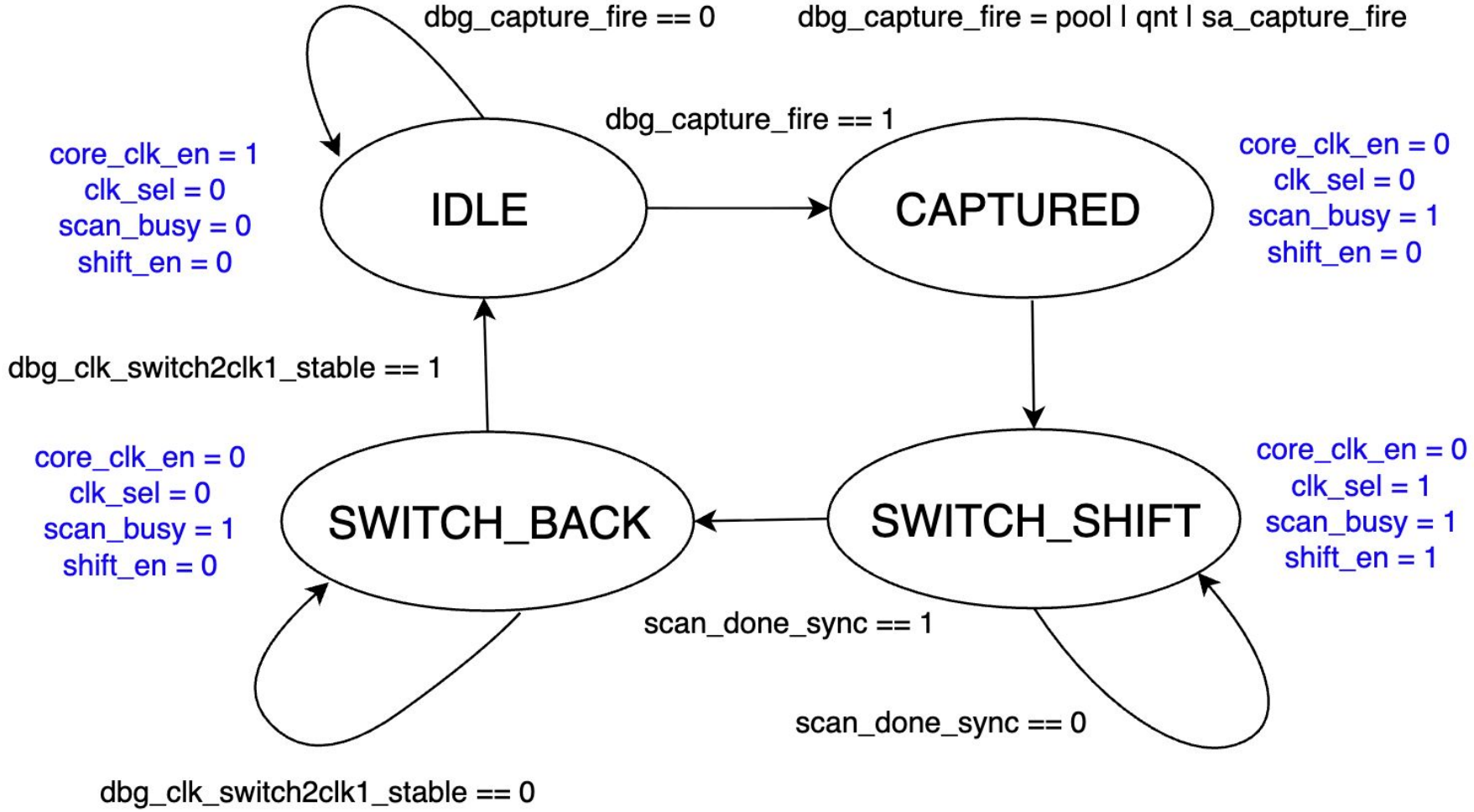
Pipeline Design for Debug

Datapath:



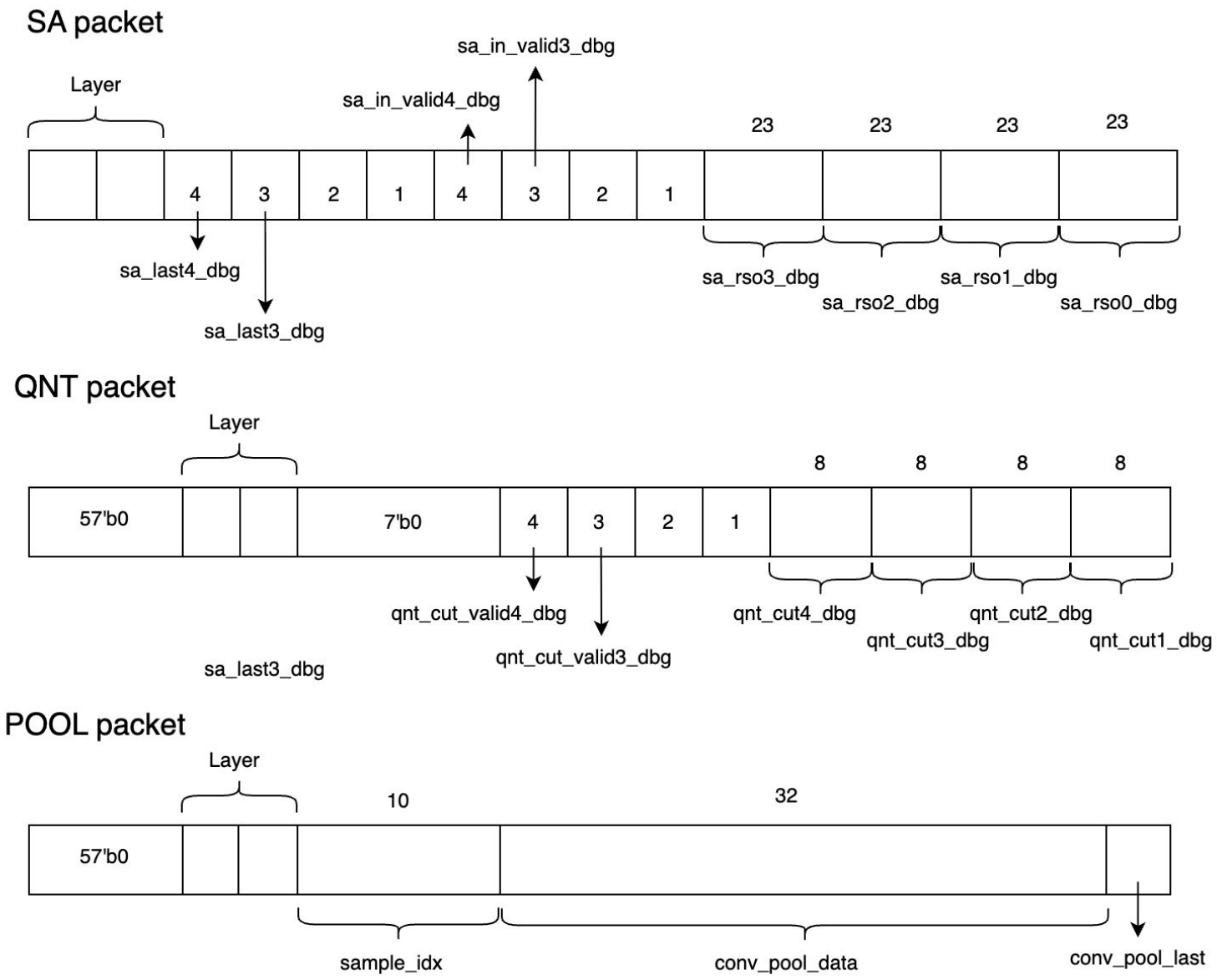
Pipeline Design for Debug

DFD fsm:



Pipeline Design for Debug

Packet Selection:



Pipeline Design for Debug

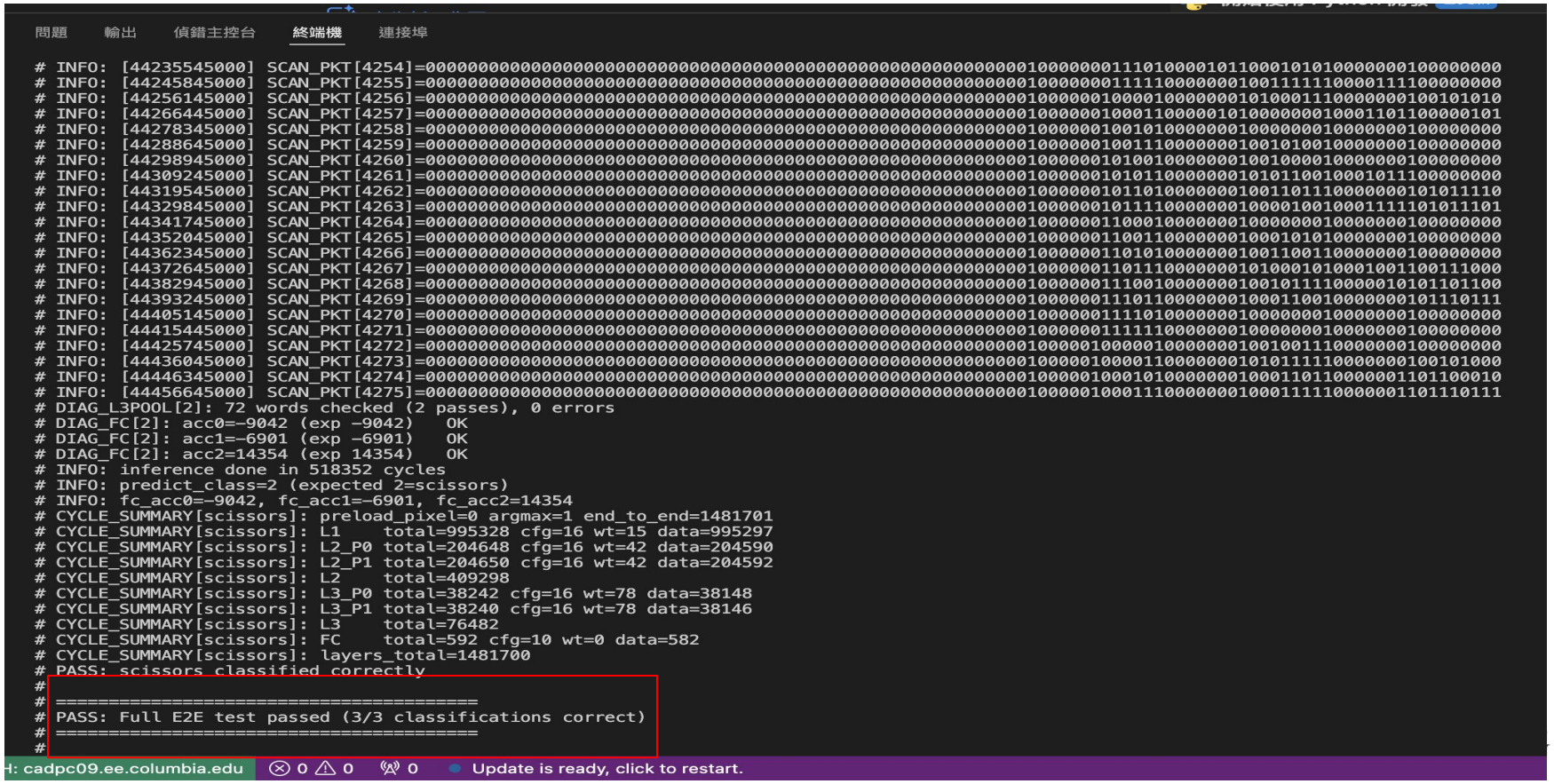
Verification - dumping:

TB_PLUSARGS="+SCAN_MODE_POOL" : dumping all pool. results

TB_PLUSARGS="+SCAN_MODE_QNT" : dumping all qnt. results

TB_PLUSARGS="+SCAN_MODE_SA" : dumping all conv. results

You can pass it through whole process, and dump all the pool results out at the same time.


問題
輸出
偵錯主控台
終端機
連接埠
INFO: [44235545000] SCAN_PKT[4254]=0000000000

Pipeline Design for Debug

Verification - comparing w MATLAB goldan model:

```
# Errors: 0, Warnings: 0
[cc5397@cadpc09 Layer_final_v2]$ python3 vf/scripts/compare_pool_scan.py
scan file   : /courses/ee6350/proj_2026Spring/team05/SAA/Layer_final_v2/tb_system_e2e_scan_out.txt
case order  : paper rock scissors
actual pkts : 4275
expect pkts : 4275
PASS: all scan packets match MATLAB pool golden data
[cc5397@cadpc09 Layer_final_v2]$
```

1) POOL PASS.

```
[cc5397@cadpc09 Layer_final_v2]$ python3 vf/scripts/compare_qnt_scan.py
scan file      : /courses/ee6350/proj_2026Spring/team05/SAA/Layer_final_v2/tb_system_e2e_scan_out.txt
case order     : paper rock scissors
packet count   : 20934
valid outputs  : 69768

== paper ==
conv1: packets=4588 lane_samples=3844 mismatches=0 layer_hist=[00:15376]
conv2: packets=2030 lane_samples=1682 mismatches=0 layer_hist=[01:6718 10:10]
  WARN: observed layer_sel outside expected 01 while collecting conv2
conv3: packets=360 lane_samples=288 mismatches=0 layer_hist=[10:1152]

== rock ==
conv1: packets=4588 lane_samples=3844 mismatches=0 layer_hist=[00:15376]
conv2: packets=2030 lane_samples=1682 mismatches=0 layer_hist=[01:6718 10:10]
  WARN: observed layer_sel outside expected 01 while collecting conv2
conv3: packets=360 lane_samples=288 mismatches=0 layer_hist=[10:1152]

== scissors ==
conv1: packets=4588 lane_samples=3844 mismatches=0 layer_hist=[00:15376]
conv2: packets=2030 lane_samples=1682 mismatches=0 layer_hist=[01:6718 10:10]
  WARN: observed layer_sel outside expected 01 while collecting conv2
conv3: packets=360 lane_samples=288 mismatches=0 layer_hist=[10:1152]

PASS: all QNT samples match MATLAB golden data (3 layer_sel warnings)
```

2) QNT PASS.

Pipeline Design for Debug

Verification - comparing w MATLAB goldan model:

```
[cc5397@cadpc09 Layer_final_v2]$ python3 vf/scripts/compare_sa_scan.py
scan file      : /courses/ee6350/proj_2026Spring/team05/SAA/Layer_final_v2/tb_system_e2e_scan_out.txt
case order     : paper rock scissors
packet count   : 20934
valid outputs  : 69768

== paper ==
conv1: packets=4588 lane_samples=3844 mismatches=0 last_counts=[248, 248, 248, 248] layer_hist=[00:15376]
conv2: packets=2030 lane_samples=1682 mismatches=0 last_counts=[116, 116, 116, 116] layer_hist=[01:6728]
conv3: packets=360 lane_samples=288 mismatches=0 last_counts=[24, 24, 24, 24] layer_hist=[10:1152]

== rock ==
conv1: packets=4588 lane_samples=3844 mismatches=0 last_counts=[248, 248, 248, 248] layer_hist=[00:15376]
conv2: packets=2030 lane_samples=1682 mismatches=0 last_counts=[116, 116, 116, 116] layer_hist=[01:6728]
conv3: packets=360 lane_samples=288 mismatches=0 last_counts=[24, 24, 24, 24] layer_hist=[10:1152]

== scissors ==
conv1: packets=4588 lane_samples=3844 mismatches=0 last_counts=[248, 248, 248, 248] layer_hist=[00:15376]
conv2: packets=2030 lane_samples=1682 mismatches=0 last_counts=[116, 116, 116, 116] layer_hist=[01:6728]
conv3: packets=360 lane_samples=288 mismatches=0 last_counts=[24, 24, 24, 24] layer_hist=[10:1152]

PASS: all SA samples match MATLAB golden data
[cc5397@cadpc09 Layer_final_v2]$
```

3) SA PASS.

Pipeline Design for Debug

STA check & area_power check:

# SETUP	WNS	TNS	FEP
View: ALL	5586.7	0.0	0
Group : C2C	8966.8	0.0	0
Group : C2O	15021.6	0.0	0
Group : cg_enable_group_clk	19332.5	0.0	0
Group : cg_enable_group_dbg_clk_clk1	15633.6	0.0	0
Group : clk	5586.7	0.0	0
Group : clk2	N/A	N/A	N/A
Group : core_clk	N/A	N/A	N/A
Group : dbg_clk_clk1	N/A	N/A	N/A
Group : dbg_clk_clk2	N/A	N/A	N/A
Group : default	N/A	N/A	N/A
Group : I2C	15606.3	0.0	0
Group : I2O	N/A	N/A	N/A

Category	Leakage	Internal	Switching	Total	Row%
memory	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
register	6.35486e-06	3.59907e-05	1.28438e-06	4.36299e-05	65.96%
latch	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
logic	1.58085e-06	4.48246e-06	4.28562e-06	1.03489e-05	15.64%
bbox	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
clock	2.18327e-07	2.07531e-06	9.87722e-06	1.21709e-05	18.40%
pad	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
pm	0.00000e+00	0.00000e+00	0.00000e+00	0.00000e+00	0.00%
Subtotal	8.15403e-06	4.25485e-05	1.54472e-05	6.61497e-05	100.00%
Percentage	12.33%	64.32%	23.35%	100.00%	100.00%

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
dft_top		284	1393.920	0.000	1393.920	ZeroWireload (S)
u_clk_gen	clk_gen	26	122.400	0.000	122.400	ZeroWireload (S)

(S) = wireload was automatically selected

Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

04 Scan Chain

05 Digital PNR Results

06 Verification

07 Summary

Parameter of Netlist

Area Breakdown (Top-Level)

Functional Block	Area (μm²)	Share (%)
Total System (system_top)	410,299.20	100.00%
Convolution / Quantization / Pooling	399,248.64	97.31%
Fully Connected Engine	6,925.32	1.69%
Top-level Control FSM	814.32	0.20%
FC Input Adapter	732.60	0.18%
SRAM-A Interface Logic	740.16	0.18%
Conv Input Adapter	608.76	0.15%
SRAM-B Interface Logic	474.12	0.12%
Layer Scheduler FSM	276.12	0.07%
Weight Pre-padding Logic	155.52	0.04%

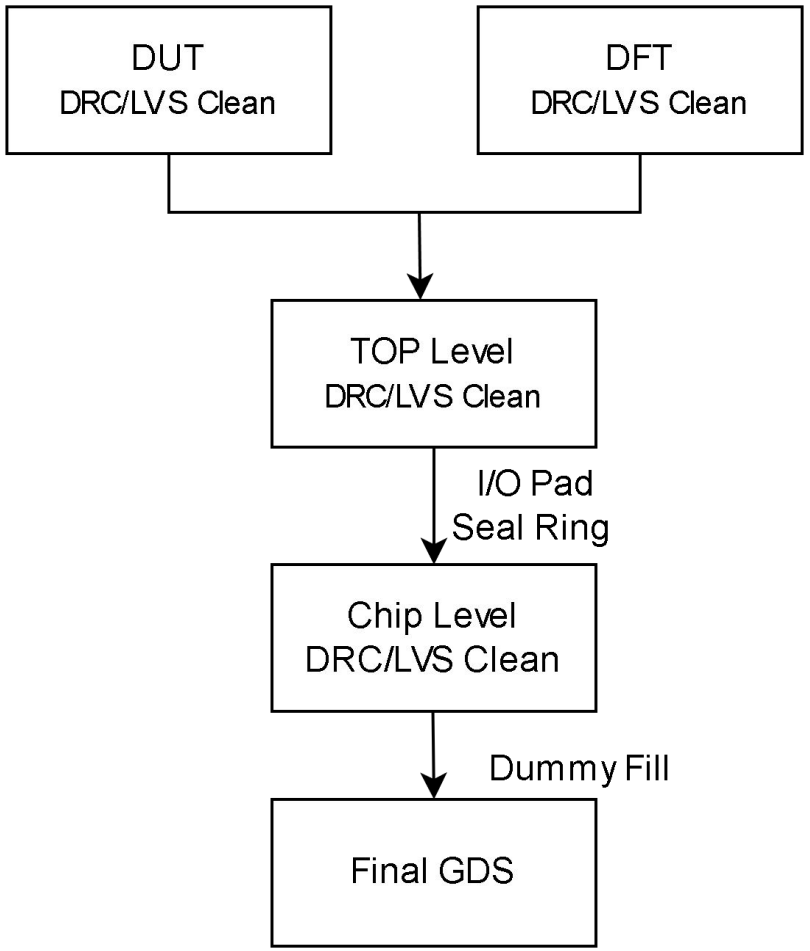
Area Breakdown inside Conv Engine

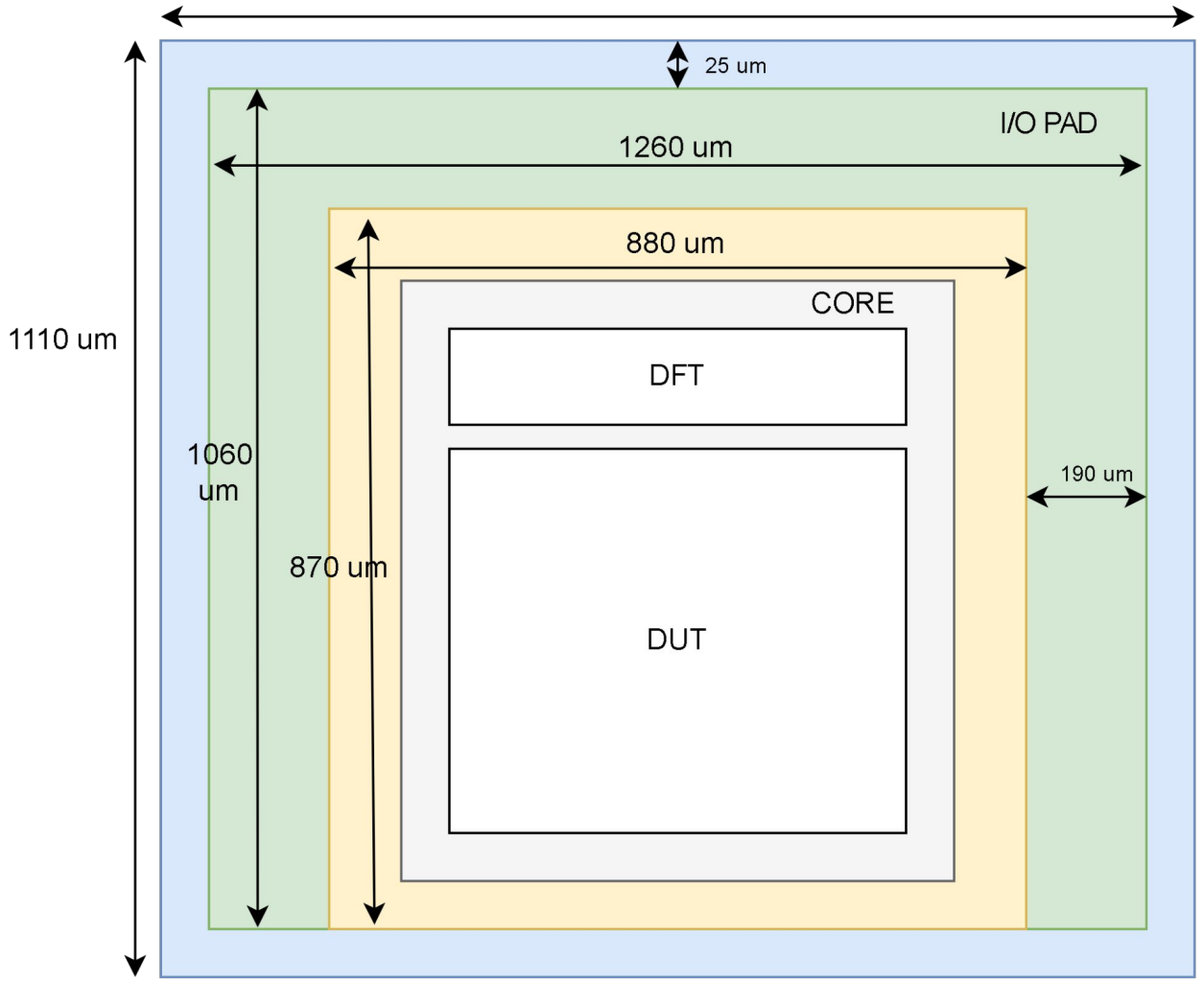
Submodule	Area (μm²)	Share in Conv (%)
Conv1 Compute Pipeline	313,878.24	78.62%
Quantization Engine	51,848.64	12.99%
Pooling Engine	27,949.68	7.00%
Quantization Param Loader	4,739.76	1.19%
Conv-to-Quant Adapter	832.32	0.21%

PAD & Layput

PNR Results

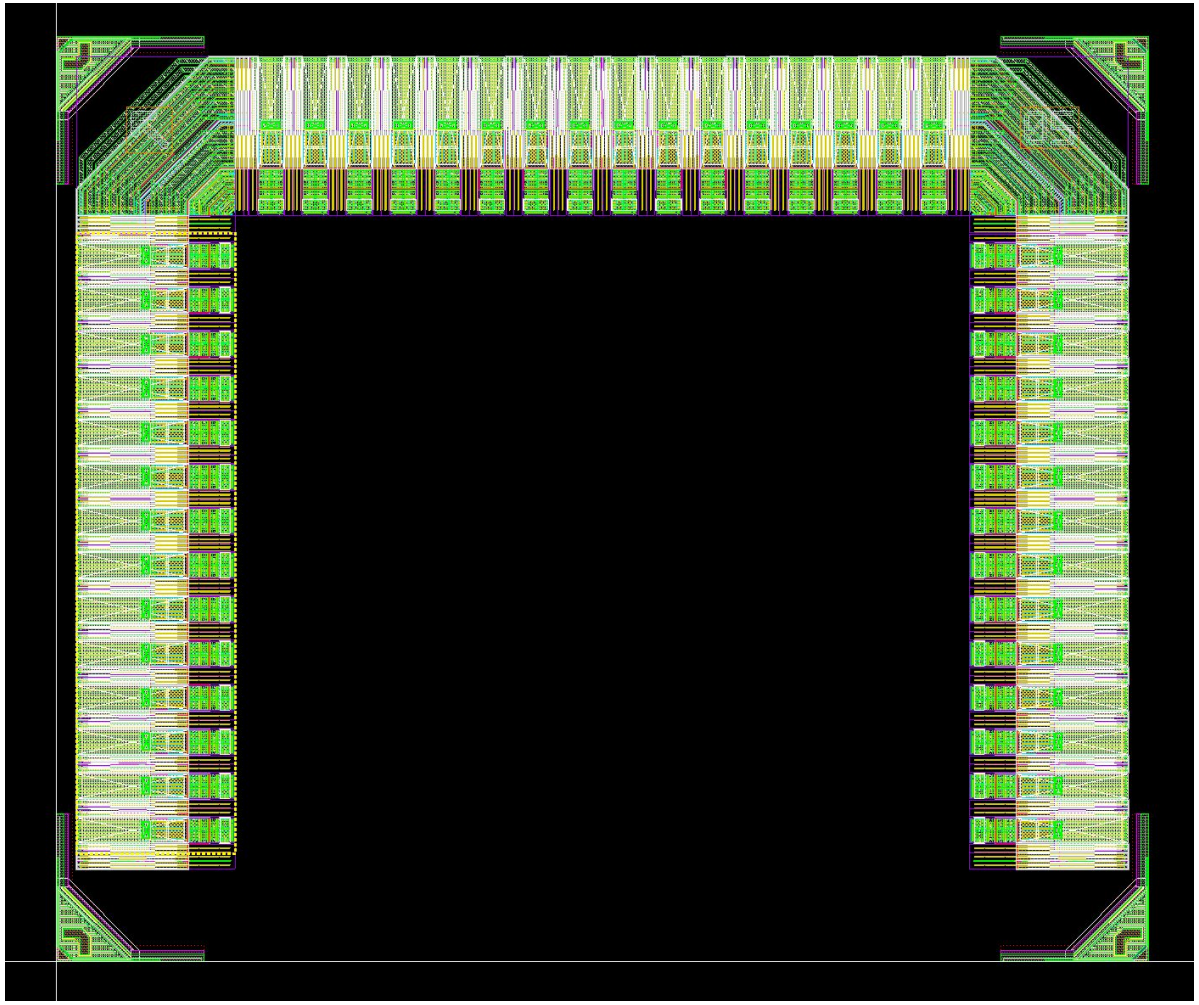
PNR FLOW





PNR Results

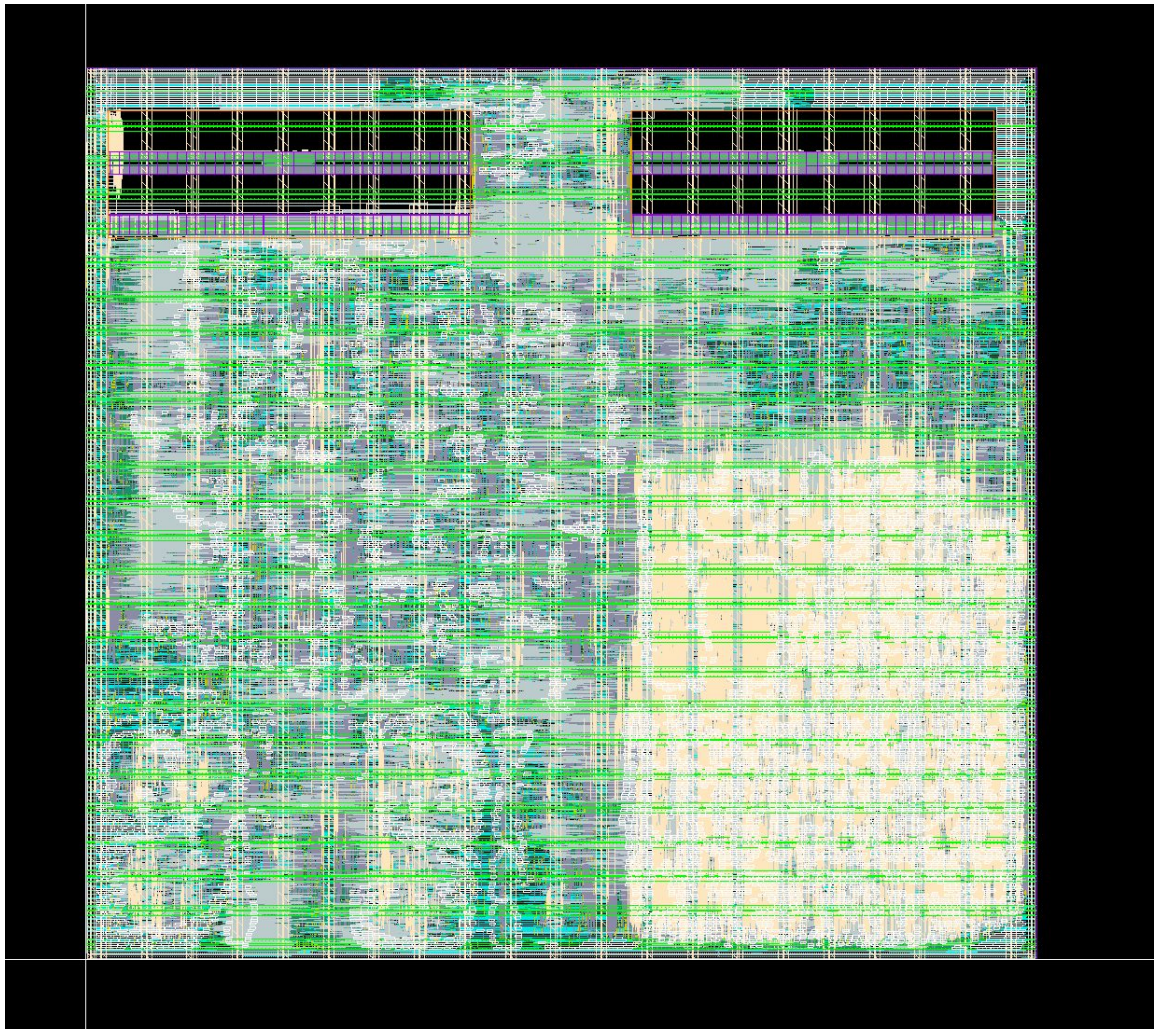
Pad Frame



PIN	Direction	Number
VSS	IN	6
VDD (1V)	IN	2
CVDD (1.1V)	IN	2
DVDD (2.5 V)	IN	2
POC	IN	1
Load_Data [15:0]	IN	16
CLK	IN	1
Load_control[2:0]	IN	3
Reset_N	IN	1
Test_Mode [1:0]	IN	2
Predict_class[1:0]	OUT	2
Scan_out	OUT	1
Scan_valid	OUT	1
Predict_valid	OUT	1
Load_ready	OUT	1
Busy	OUT	1

PNR Results

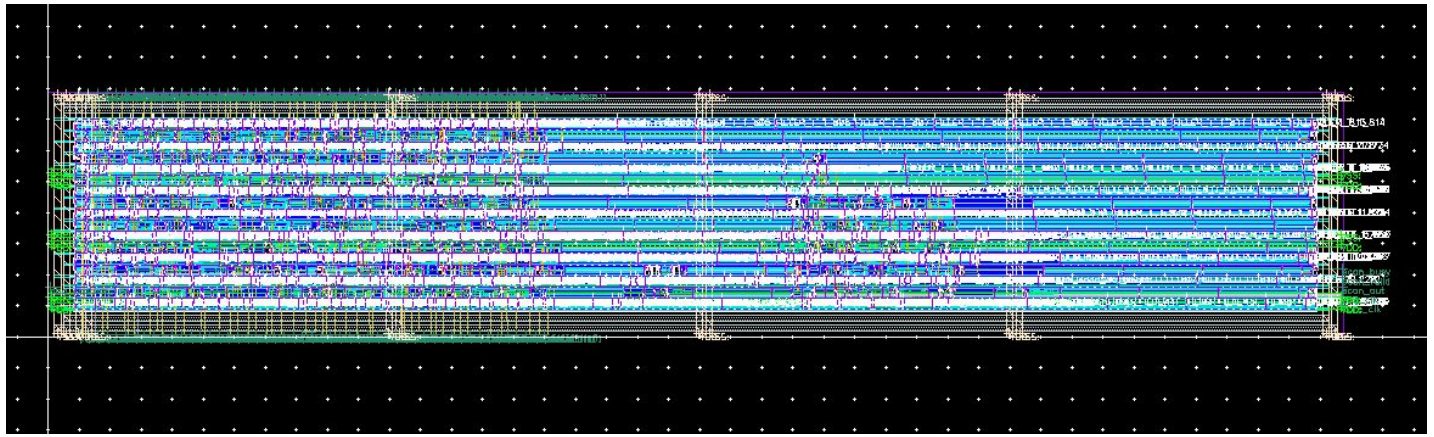
DUT



Metric	Value
Core Area	627,342 µm²
Chip Area	656,092 µm²
Core Density	78.8%
Worst Setup Slack @50MHz	+8.6 ns
Worst Hold Slack	+0.10 ns
DRC	Clean (not including DN)
LVS	Clean

PNR Results

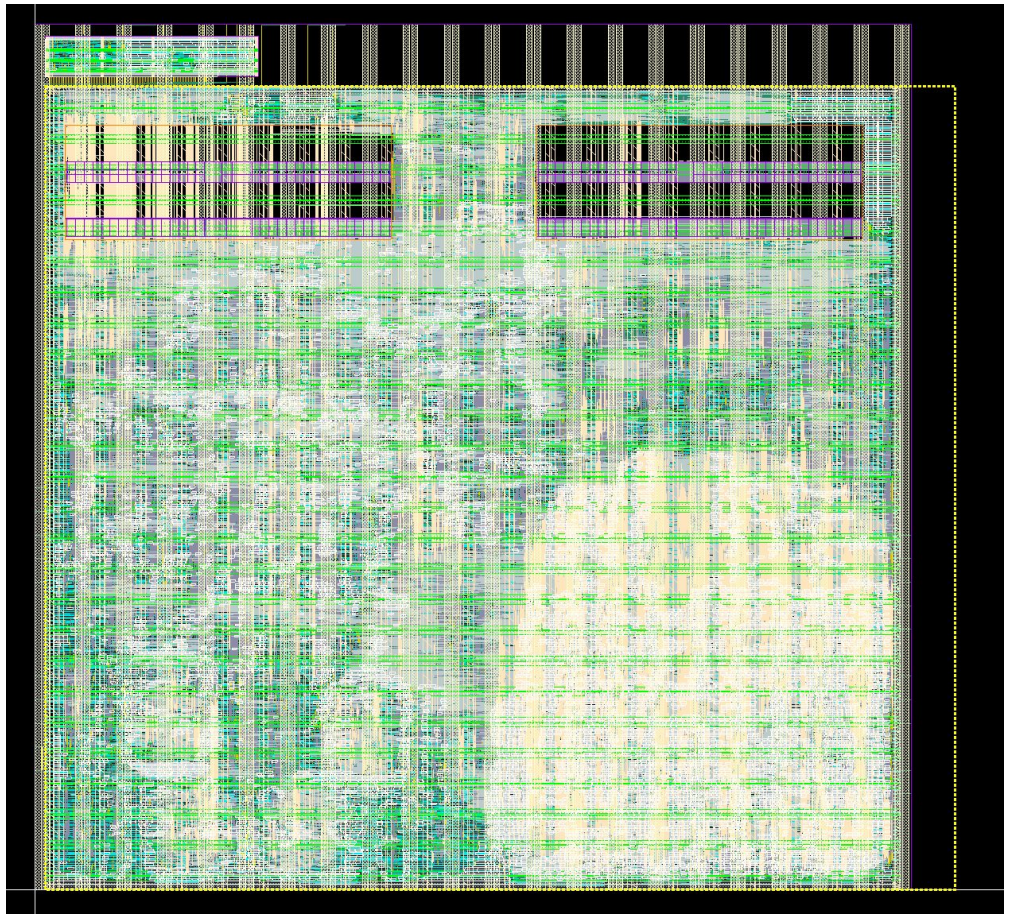
DFT

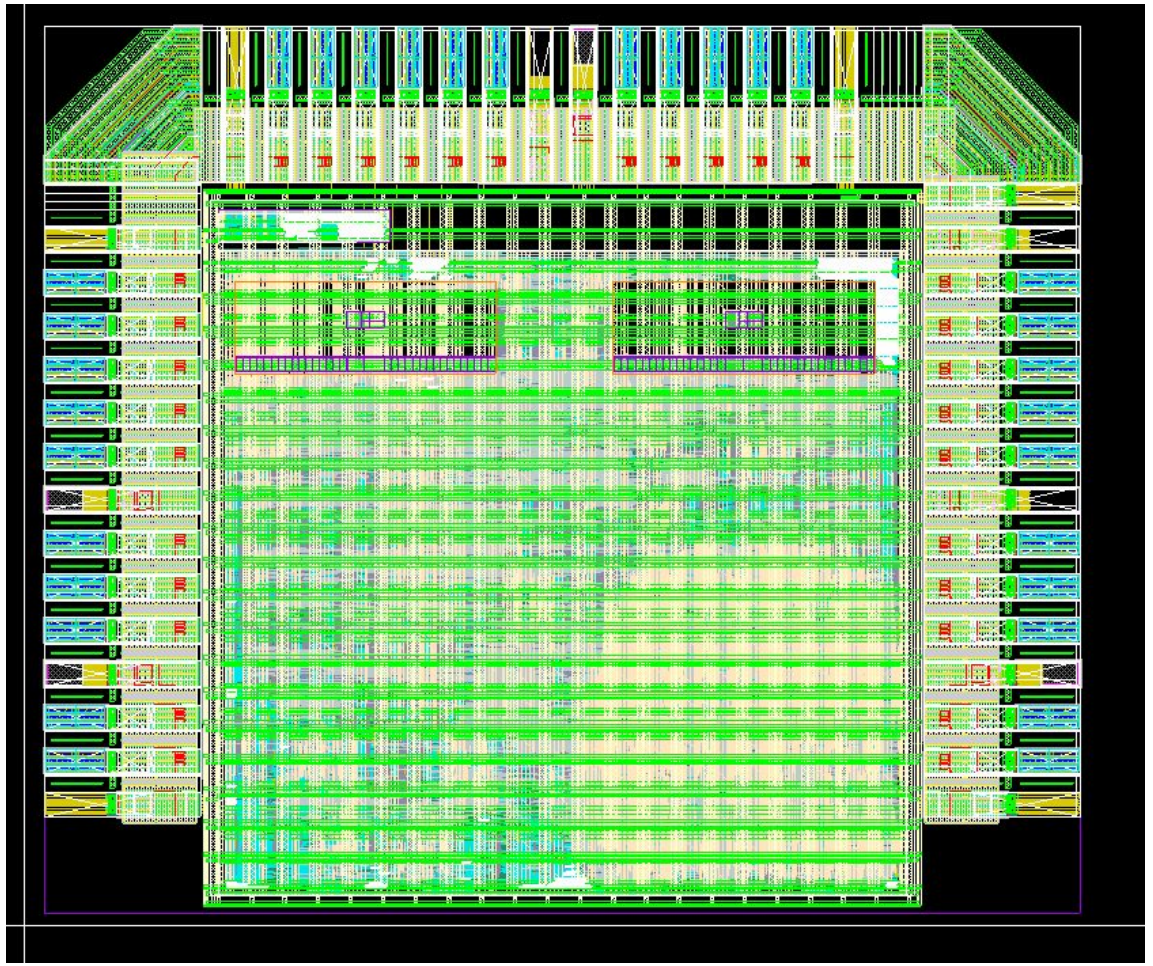


Metric	Value
Core Area	4896 μm²
Chip Area	6650 μm²
Core Density	40%
Worst Setup Slack	+5.3ns
Worst Hold Slack	+0.1ns
DRC	CLEAN
LVS	CLEAN

PNR Results

top -chip





PNR Results

FINAL GDS

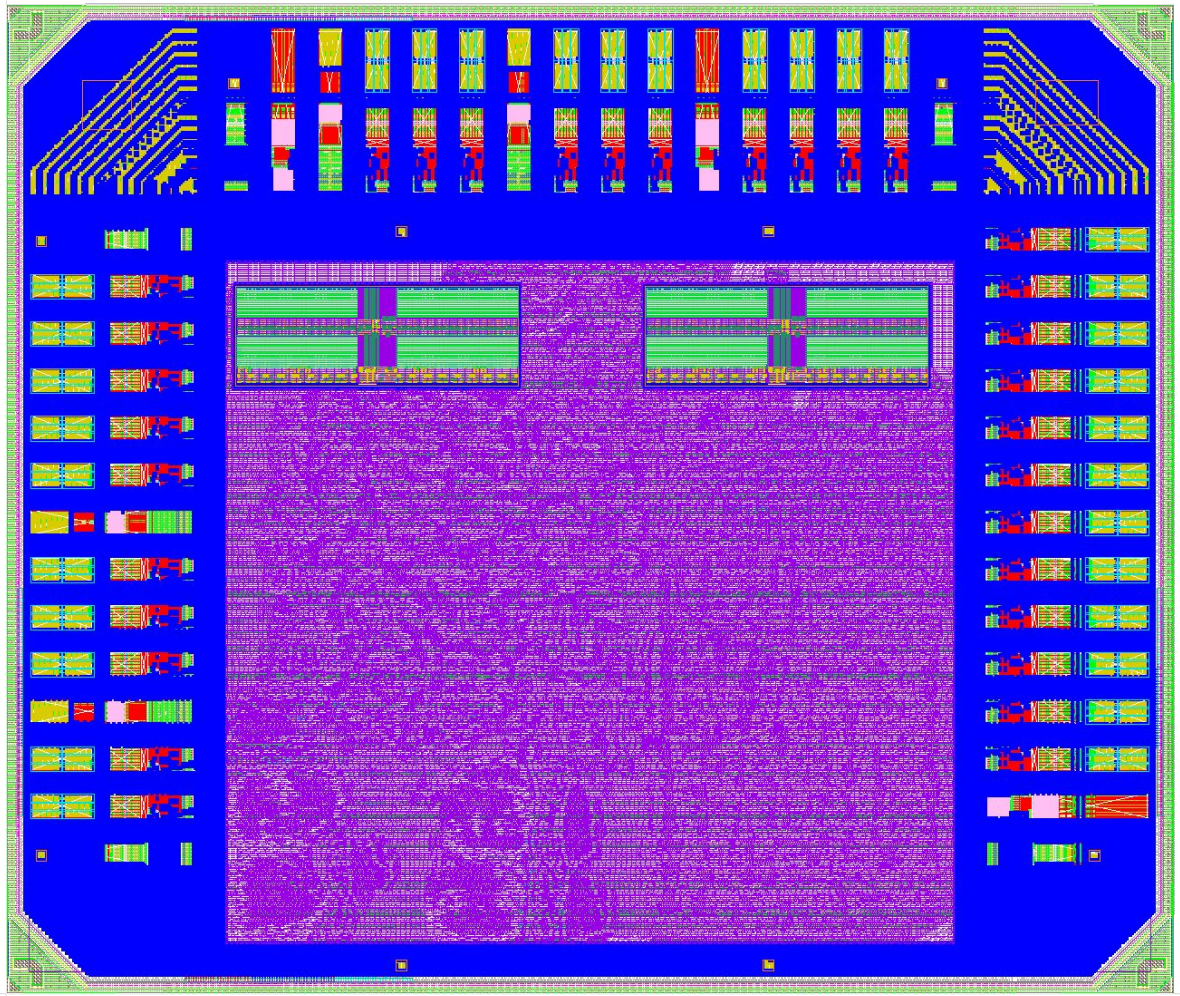


Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

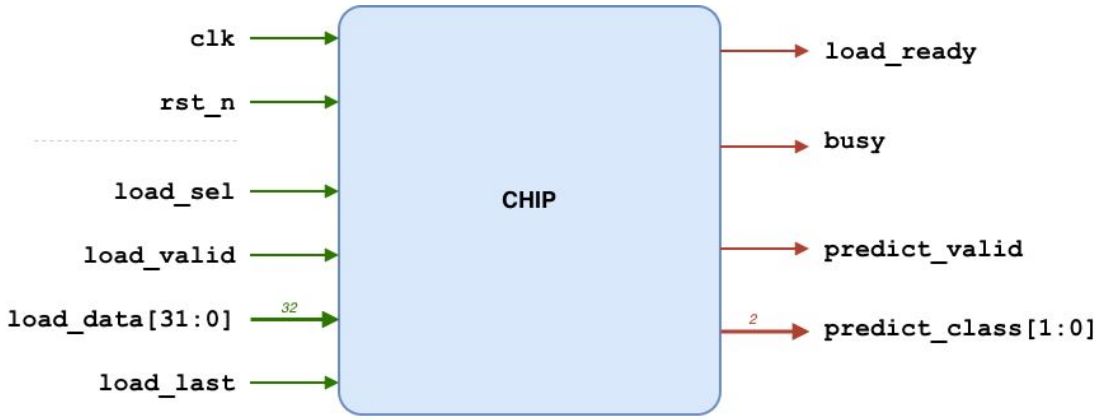
04 Scan Chain

05 Digital PNR Results

06 Verification

07 Summary

Behavior Simulation (3 cases)



	PAPER	ROCK	SCISSORS
Layer 1 output	961/961 √ (31x31x4/4)	961/961 √	961/961 √
Layer 2 group1 output	196/196 √ (14x14x4/4)	196/196 √	196/196 √
Layer 2 group2 output	196/196 √	196/196 √	196/196 √
Layer 3 group1 output	36/36 √ (6x6x4/4)	36/36 √	36/36 √
Layer 3 group2 output	36/36 √	36/36 √	36/36 √
Fully Connection	3 channel √	3 channel √	3 channel √
Predicted class	√	√	√

Load model	cfg	weight	Total
cycles	99	1031	1130

Inference	Layer1	Layer2	Layer3	FC	Total
cycles	6332	6040	2400	592	15396

Behavior Simulation (all images in testing dataset)

Test 1679 images:

Matlab golden module agreement:
100% in every layers' output

Classify Accuracy: 94.40%
(the same as the golden module)

Class	Accuracy
paper	92.08 % (535/581)
rock	99.26 % (533/537)
scissors	92.16 % (517/561)
Overall	94.40 % (1,585/1,679)

Post PNR Simulation @50MHz

---- Post-PnR netlist
(no physical-only cells, has VDD/VSS in module ports)
1.system_top/system_top.PG.v

---- SRAM macros ----
sram_A.v sram_B.v

---- SDF ----
system_top.verilog.sdf

Gate-level simulation was performed across three PVT corners using the corresponding SDF files:

- **Min corner (`-sdfmin`, FF / fast):** verifies **hold timing** — ensures no race conditions when paths are fastest.
- **Typ corner (`-sdftyp`, TT / nominal):** functional regression under typical operating conditions.
- **Max corner (`-sdfmax`, SS / slow):** verifies **setup timing** — ensures the design meets target frequency under worst-case delay.

All three cases passed without timing violations

```
# === TEST: MODEL_LOAD ===
# INFO: sending CFG segment (48 words)...
# INFO: sending WEIGHT segment (513 words)...
#
# === TEST: paper INFERENCE ===
# INFO: sending paper image for inference...
# INFO: inference done in 9110 cycles
# INFO: predict_class=00 (expected 0=paper)
# PASS: paper classified correctly
#
# === TEST: rock INFERENCE ===
# INFO: sending rock image for inference...
# INFO: inference done in 9110 cycles
# INFO: predict_class=01 (expected 1=rock)
# PASS: rock classified correctly
#
# === TEST: scissors INFERENCE ===
# INFO: sending scissors image for inference...
# INFO: inference done in 9110 cycles
# INFO: predict_class=10 (expected 2=scissors)
# PASS: scissors classified correctly
#
# ========================================
# PASS: Post-PnR E2E test passed (3/3 classifications correct)
# ========================================
```

Table of Contents

01 Introduction

02 Demo Notes

03 Block Diagram and Dataflow

04 Scan Chain

05 Digital PNR Results

06 Verification

07 Summary

Block	Function	RTL	Syn	PNR
Systolic Array	CNN convolution MAC array	✓	✓	✓
Bank Buffer	Feature map buffering between layers	✓	✓	✓
Weight Buffer	Store convolution weights	✓	✓	✓
SRAM Wrapper	Interface between SRAM macro and logic	✓	✓	✓
8KB SRAM	Feature map storage	✓	✓	✓
4KB SRAM	Weight / intermediate storage	✓	✓	✓
Address Generator	Generate sequential read/write addresses	✓	✓	✓
Unpacker	Convert 32-bit SRAM word to 8-bit pixel stream	✓	✓	✓
Pooling	Downsample feature maps (MaxPool)	✓	✓	✓
Quantization	Convert higher precision to INT8	✓	✓	✓
Line Buffer	Store rows for convolution window	✓	✓	✓
CNN Control FSM	Control CNN pipeline execution	✓	✓	✓
Window Buffer	Generate convolution sliding window	✓	✓	✓
DFT FSM	Control scan/test operations	✓	✓	✓
DFT Scan Chain	Scan chain for testing	✓	✓	✓
Core	Integrate all modules(W/O DFT)	✓	✓	✓
CORE + DFT	Integrate CORE and DFT module	✓	✓	WIP

THE END

Thankyou for listening!

Team 5: LIC Lab
Instructor: Prof. Harish Krishnaswamy
Teaching Assistant: Alfred Davidson
Group member: Chengcheng Xu(cx2355); Linxiao Wu(lw3227); Yizheng Tang(yt2992); Cheng-Wen Chu(cc5397)
Date:05/01/2026

COLUMBIA ENGINEERING